

Otulia Web: Deep Dive - Chapter 6: Asset Management & Cloudinary

6.1 Introduction to Asset Management

Luxury marketplaces are fundamentally visual. The "White Glove" experience depends on high-resolution images, video tours, and verified documents. To handle this without slowing down our own servers, Otulia uses **Cloudinary**—a global Content Delivery Network (CDN) specialized in media optimization.

6.2 Cloudinary Infrastructure

6.2.1 Configuration (`cloudinary.js`)

We centralize our media keys and storage logic to ensure every image is uploaded with the same quality standards.

```
// 1. Connection to the Cloudinary CDN
cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET,
});

// 2. Intelligent Storage: Automatically decides where to save the file
const storage = new CloudinaryStorage({
  cloudinary: cloudinary,
  params: async (req, file) => {
    // 3. User-Based Isolation: Every agent gets their own private folder
    const folders = getFolderPaths(req.user.email);

    return {
      folder: folders.assets, // Organization: otulia_assets
      allowed_formats: ['jpg', 'png', 'jpeg', 'pdf'], // Security: only allow safe formats
      resource_type: 'auto', // Detects if it's an Image or a PDF document automatically
    };
  },
});
```

6.3 Organized Folder Logic (`cloudinaryFolders.js`)

We use a "Sanitization" strategy to turn user emails into safe folder names. This prevents illegal characters from breaking our media links.

```
const getFolderPaths = (userEmail) => {
  // Turns "admin@otulia.com" into "admin_otulia_com"
  const sanitizedEmail = userEmail ? userEmail.replace(/[@]/g, '_').replace(/[.]/g, '_') :
  'anonymous';

  return {
```

```

    verification: `verification/${sanitizedEmail}`, // KYC Documents
    profile: `users/${sanitizedEmail}/profile`, // Profile Picture
    company: `users/${sanitizedEmail}/company`, // Company Logos
    assets: `otulia_assets`, // Public Marketplace Images
  };
};

```

6.4 The Upload Lifecycle

Let's follow a file from the User's computer to the global CDN:

1. **Selection:** The user selects a high-res photo of a Yacht.
2. **Frontend:** The React component creates a `FormData` object.
3. **Transport:** The file is sent to the backend via a `POST` request.
4. **Middleware: Multer** intercepts the request. Instead of saving to our hard drive, it "streams" the file directly to Cloudinary.
5. **CDN Confirmation:** Cloudinary sends back a `secure_url` (e.g., `https://res.cloudinary.com/otulia/image/upload/v123/yacht.jpg`).
6. **Database:** We save only this **URL string** in MongoDB. We never store the actual file in the database.

6.5 Real-Time Data Synchronization (`assetUpdater.js`)

Luxury listings are "Denormalized" for speed. This means the Agent's logo is stored inside every car listing they post. If the agent changes their logo, we must update every listing they ever posted to ensure consistency.

```

const updateUserAssetsAgent = async (userId, updateData) => {
  const models = [CarAsset, BikeAsset, YachtAsset, EstateAsset, Listing];

  // 1. Prepare the Update: Target specific agent fields
  const updateQuery = {};
  if (updateData.companyLogo) updateQuery['agent.companyLogo'] = updateData.companyLogo;
  if (updateData.photo) updateQuery['agent.photo'] = updateData.photo;

  // 2. Parallel Processing: Run updates on all collections at once
  const updatePromises = models.map(Model =>
    Model.updateMany(
      { 'agent.id': userId.toString() }, // Only target this specific agent
      { $set: updateQuery } // Apply the new photo/logo
    )
  );

  await Promise.all(updatePromises); // Wait for all 5 categories to finish
};

```

6.6 Optimized Media Rendering

When the frontend displays an image, it uses Cloudinary's "On-the-Fly" transformations:

- **Optimization:** The images are automatically compressed to the smallest possible file size without losing "Luxury Quality."
- **Formatting:** Cloudinary detects the user's browser. If they are on Chrome, it serves a `.webp` file; if on an old iPhone, it serves a `.jpg`.
- **Security:** For sensitive "Verification Documents," we use the `resource_type: 'image'` and Cloudinary SDK to generate "Secure URLs" that expire, ensuring private documents aren't public forever.